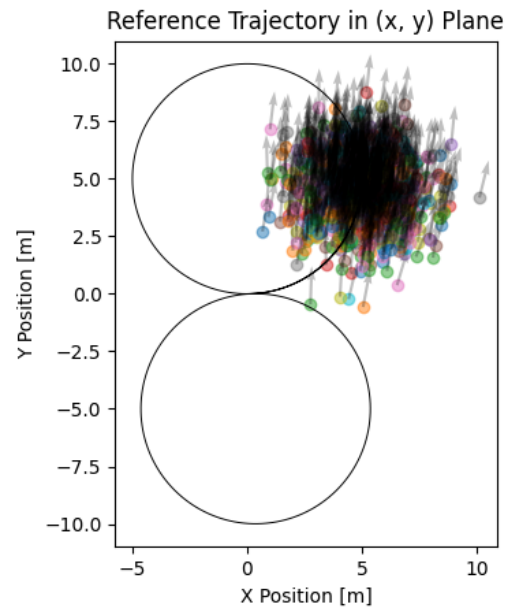


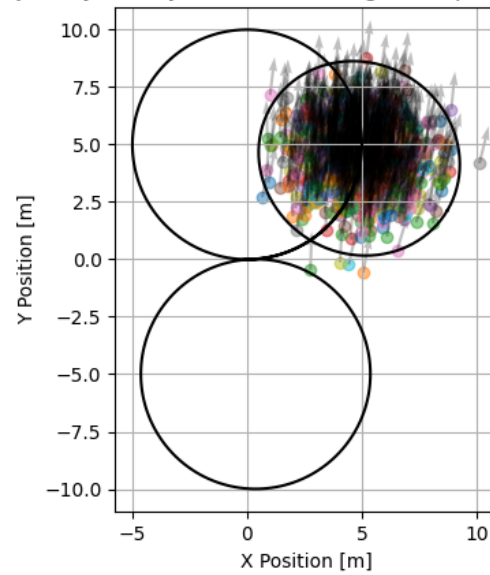
1.b)

Figure 1: Reference Trajectory



1.c)

Figure 2: Gaussian Fit

Trajectory in (x, y) Plane with 3 Sigma Ellipse at $T = 35$ s

1.e)

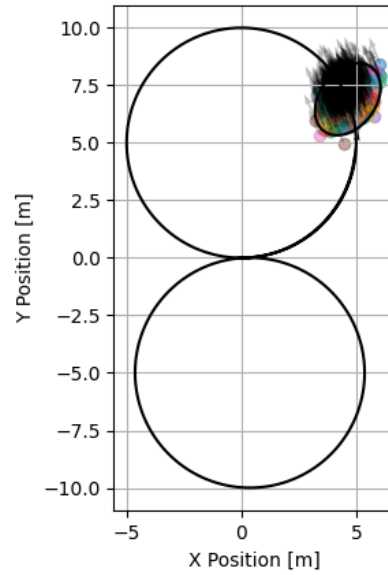
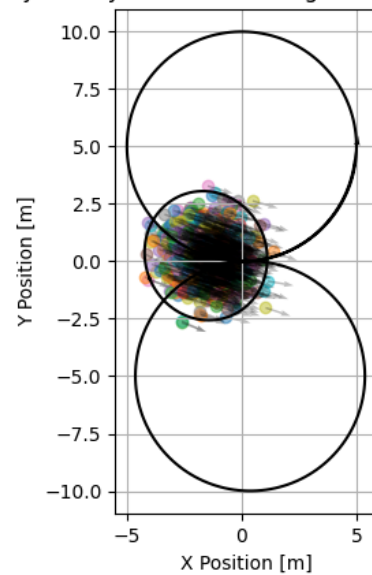
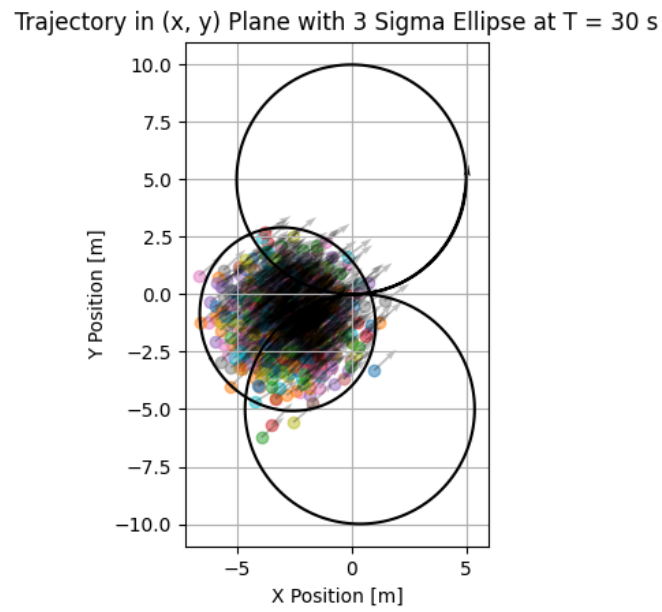
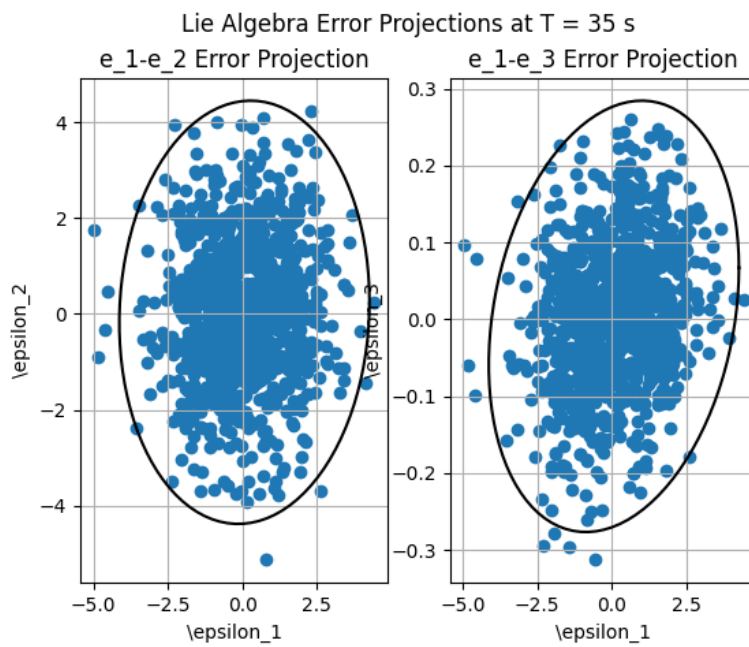
Figure 3: Distribution at $t = 5$ sTrajectory in (x, y) Plane with 3 Sigma Ellipse at $T = 5$ sFigure 4: Distribution at $t = 15$ sTrajectory in (x, y) Plane with 3 Sigma Ellipse at $T = 15$ s

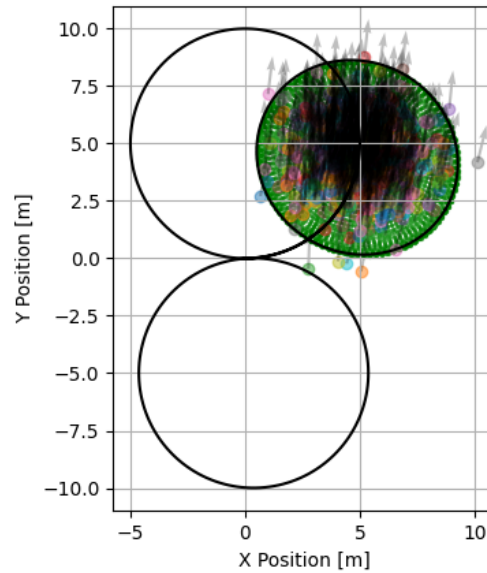
Figure 5: Distribution at $t = 30$ s

1.f)

Figure 6: Distribution in Lie Algebra



1.g)

Figure 7: Distribution at $t = 35$ s Projected from Lie AlgebraTrajectory in (x, y) Plane with 3 Sigma Ellipse at $T = 35$ s

Code Snippet 1: Problem Data Code, inputpath=../..../HW4

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from parfor import pmap
4 from os import environ
5 environ['OMP_NUM_THREADS'] = '4'
6 from Code.SE2.SE2_maps import se2_compose, se2_exp, se2_log, se2_wedge, se2_vee, se2_inverse
7 from Code.SE2.SE2_integrators import se2_euler_integrator, se2_Lie_group_integrator
8
9 class AAE590LGM_HW4_Problem:
10     # Simulation
11     delta_t = 0.1 # [s] Time step
12     T = 35 # [s] Simulation duration
13
14     # Reference trajectory
15     v_nom = 2 # [m / s] Commanded forward speed
16     r_fig8 = 5 # [m] Radius of each lobe of the figure-eight
17     omega_turn = 0.4 # [rad / s] Magnitude of the commanded heading rate on each lobe
18     T_fig8 = 10 * np.pi # [s] Period of one figure-eight cycle
19     T_lobe = T_fig8 / 2 # [s] Period of one lobe
20
21     # Process noise
22     sigma_v = 1 # [m / s] Forward speed standard deviation
23     sigma_lat = 0 # [m / s] Lateral speed standard deviation (0 so no sideslip)
24     sigma_omega = 0.05 # [rad / s] Heading rate standard deviation
25
26     # GPS measurement model parameters
27     sigma_GPS = 0.3 # [m] GPS measurement noise standard deviation
28     GPS_rate = 1 # [Hz] GPS measurements arrive once per second
29
30     rng = np.random.default_rng(2)
31     N_steps = np.round(T / delta_t).astype(int)
32     N_nodes = N_steps + 1
33     t_k = np.linspace(0, T, N_nodes)

```

```

34
35     v_x = v_nom * np.ones((1, N_nodes))
36     v_y = np.zeros((1, N_nodes)) # No sideslip
37     tau = np.mod(t_k, T_fig8)
38     omega = np.where(tau < T_lobe, omega_turn, -omega_turn)
39     xi_k = np.vstack((v_x, v_y, omega))
40
41     Q_body = np.diag(np.square([sigma_v, 0, sigma_omega]))
42
43     ## Create nominal reference trajectory (figure 8)
44     def simulate(self, xi_noise):
45         X_0 = np.eye(3)
46
47         X_k = np.zeros([3, 3, self.N_nodes])
48         X_k[:, :, 0] = X_0
49
50         for k in range(self.N_steps):
51             X_k[:, :, k + 1] = se2_Lie_group_integrator(X_k[:, :, k], self.xi_k[:, k] +
52             xi_noise[:, k], self.delta_t)
53
54         return X_k
55
56     def monte_carlo(self, N_runs):
57         X_k_runs = np.zeros((3, 3, self.N_nodes, N_runs))
58
59         xi_noise = np.vstack((self.rng.normal(0, self.sigma_v, size = (1, self.N_steps,
60         N_runs)),
61                                np.zeros((1, self.N_steps, N_runs)),
62                                self.rng.normal(0, self.sigma_omega, size = (1, self.N_steps,
63         N_runs))))
64
65         for i in range(N_runs):
66             X_k_runs[:, :, :, i] = self.simulate(np.squeeze(xi_noise[:, :, i]))
67
68         return X_k_runs
69
70     def monte_carlo_multithreaded(self, N_runs):
71         X_k_runs = np.zeros((3, 3, self.N_nodes, N_runs))
72
73         xi_noise = np.vstack((self.rng.normal(0, self.sigma_v, size = (1, self.N_steps,
74         N_runs)),
75                                np.zeros((1, self.N_steps, N_runs)),
76                                self.rng.normal(0, self.sigma_omega, size = (1, self.N_steps,
77         N_runs))))
78
79         # issue tasks and gather results
80         sim_func = lambda i: self.simulate(np.squeeze(xi_noise[:, :, i]))
81         results = pmap(sim_func, range(N_runs))
82
83         for i in range(N_runs):
84             X_k_runs[:, :, :, i] = results[i]
85
86         return X_k_runs

```

Code Snippet 2: The Banana Distribution Code

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import time
4 from Code.SE2.SE2_maps import se2_compose, se2_exp, se2_log, se2_wedge, se2_vee, se2_inverse
5 from Code.SE2.SE2_integrators import se2_euler_integrator, se2_Lie_group_integrator
6 from Code.Plotting.covariance_plotting import plot_cov_ellipse, calc_and_plot_cov_ellipse,
7     cov_ellipse_3D
8 from Code.Plotting.SE2_plotting import plot_scatter_unicycles,
9     plot_scatter_unicycles_with_cov, plot_Lie_algebra_cov_ellipse
10 # Problem 1: The Banana Distribution

```

```

11
12 ## Unicycle SE2 Dynamics
13 # Xdot = X xi^hat
14 # xi = [v; 0; omega]
15
16 ## Initialize Problem Parameters
17 prob = AAE590LGM_HW4_Problem()
18
19 ## 1a) Simulate the Unicycle
20 # Sim
21 t_ref = prob.t_k
22 X_ref = prob.simulate(np.zeros((3, prob.N_nodes)))
23
24 # Monte Carlo
25 N_runs = 1000
26 start = time.perf_counter()
27 #X_runs = prob.monte_carlo(N_runs)
28 X_runs = prob.monte_carlo_multithreaded(N_runs)
29 end = time.perf_counter()
30 print(f"Elapsed: {(end - start) * 1e3:.3f} ms")
31
32 ## 1b) Plot the Distribution
33
34 plot_scatter_unicycles(X_ref, X_runs, False)
35 plt.title("Reference Trajectory in (x, y) Plane")
36 plt.gca().set_aspect('equal', adjustable = 'box')
37 plt.grid()
38 plt.savefig("./HW4/Parts/Q1/AAE590LGM_HW4_Q1b_ref.png")
39 plt.show()
40
41 ## 1c) Fit a Gaussian
42
43 # Compute sample mean and covariance of final positions
44 # Plot 3 sigma ellipse on top of scatter
45
46 plot_scatter_unicycles_with_cov(X_ref, X_runs, False)
47 plt.title("Trajectory in (x, y) Plane with 3 Sigma Ellipse at T = 35 s")
48 plt.gca().set_aspect('equal', adjustable = 'box')
49 plt.grid()
50 plt.savefig("./HW4/Parts/Q1/AAE590LGM_HW4_Q1c_gfit.png")
51 plt.show()
52
53
54 ## 1e) Intermediate Snapshots
55
56 plot_scatter_unicycles_with_cov(X_ref, X_runs[:, :, :50, :], False)
57 plt.title("Trajectory in (x, y) Plane with 3 Sigma Ellipse at T = 5 s")
58 plt.gca().set_aspect('equal', adjustable = 'box')
59 plt.grid()
60 plt.savefig("./HW4/Parts/Q1/AAE590LGM_HW4_Q1e_t5.png")
61 plt.show()
62
63 plot_scatter_unicycles_with_cov(X_ref, X_runs[:, :, :150, :], False)
64 plt.title("Trajectory in (x, y) Plane with 3 Sigma Ellipse at T = 15 s")
65 plt.gca().set_aspect('equal', adjustable = 'box')
66 plt.grid()
67 plt.savefig("./HW4/Parts/Q1/AAE590LGM_HW4_Q1e_t15.png")
68 plt.show()
69
70 plot_scatter_unicycles_with_cov(X_ref, X_runs[:, :, :300, :], False)
71 plt.title("Trajectory in (x, y) Plane with 3 Sigma Ellipse at T = 30 s")
72 plt.gca().set_aspect('equal', adjustable = 'box')
73 plt.grid()
74 plt.savefig("./HW4/Parts/Q1/AAE590LGM_HW4_Q1e_t30.png")
75 plt.show()
76
77
78 ## 1f) Lie Algebra Gaussian Fit

```

```

79 # Calculate error w.r.t. reference trajectory
80 error = np.zeros((3, N_runs))
81 for i in range(N_runs):
82     error[:, i] = np.reshape(se2_log(se2_compose(se2_inverse(X_ref[:, :, -1]), X_runs[:, :,
83         -1, i])), (3,))
84
85 # Plot e1-e2 and e1-e3 projections
86 plt.subplot(1, 2, 1)
87 e12_mean, e12_std, e12_cov = calc_and_plot_cov_ellipse(error[0:2, :])
88 plt.scatter(error[0, :], error[1, :])
89 plt.xlabel("\epsilon_1")
90 plt.ylabel("\epsilon_2")
91 plt.title("e1-e2 Error Projection")
92 plt.grid()
93
94 plt.subplot(1, 2, 2)
95 e13_mean, e13_std, e13_cov = calc_and_plot_cov_ellipse(error[[0, 2], :])
96 plt.scatter(error[0, :], error[2, :])
97 plt.xlabel("\epsilon_1")
98 plt.ylabel("\epsilon_3")
99 plt.title("e1-e3 Error Projection")
100 plt.grid()
101
102 plt.suptitle("Lie Algebra Error Projections at T = 35 s")
103 plt.savefig("./HW4/Parts/Q1/AAE590LGM_HW4_Q1f_liealg.png")
104 plt.show()
105
106 ## 1g) Exponentiate the Ellipsoid Boundary
107 error_cov = np.cov(error)
108 plot_Lie_algebra_cov_ellipse(error_cov, X_ref[:, :, -1], "green", "-")
109 plot_scatter_uncycles_with_cov(X_ref, X_runs, False)
110 plt.title("Trajectory in (x, y) Plane with 3 Sigma Ellipse at T = 35 s")
111 plt.grid()
112 plt.gca().set_aspect('equal', adjustable = 'box')
113 plt.savefig("./HW4/Parts/Q1/AAE590LGM_HW4_Q1g_t35.png")
114 plt.show()

```